

**Definir Estándares De Codificación, De Acuerdo Con La Plataforma De Desarrollo
Elegida**

GA7-220501096-AA1-EV02

Autor:

- Yeison Adolfo Díaz Tapasco

Institución:

Servicio Nacional de Aprendizaje (SENA)

Programa: Análisis y Desarrollo de Software

Profesor:

Oscar Fernando Carvajal

Abril 2026

TABLA DE CONTENIDO

- 1. Introducción
- 2. Objetivos
 - 2.1 Objetivo General
 - 2.2 Objetivos Específicos
- 3. Descripción del Software
- 4. Definición y Aplicación del Paradigma de Programación
 - 4.1 Definición del paradigma
 - 4.2 Justificación
 - 4.3 Aplicación en el sistema
- 5. Estándares de Codificación
 - 5.1 Convención de Nombres
 - 5.2 Reglas Obligatorias de Codificación
 - 5.3 Manejo de Errores
 - 5.4 Estructura del Proyecto
 - 5.5 Convenciones del Proyecto
 - 5.6 Buenas Prácticas de Desarrollo
- 6. Aplicación de Estándares en el Sistema
 - 6.1 Funcionalidad 1: Registro de Vehículos
 - 6.2 Funcionalidad 2: Cálculo de Tarifa
 - 6.3 Funcionalidad 3: Generación de Reportes
- 7. Evidencias (Pantallazos)
- 8. Conclusiones

1. INTRODUCCIÓN

En el desarrollo de software, la implementación de estándares de codificación es esencial para garantizar la calidad, legibilidad, mantenibilidad y escalabilidad del sistema. Estos estándares permiten que el código sea uniforme, comprensible y fácil de mantener por diferentes desarrolladores.

El presente informe técnico define los estándares de codificación que serán aplicados en el desarrollo de un sistema de gestión de parqueadero, estableciendo reglas claras sobre nomenclatura, estructura del código, manejo de errores y buenas prácticas.

2. OBJETIVOS

2.1 Objetivo General

Definir e implementar estándares de codificación adecuados para el desarrollo del sistema de parqueadero, garantizando calidad, claridad y mantenibilidad del software.

2.2 Objetivos Específicos

- Establecer reglas claras para nombramiento de variables, funciones y clases.
- Aplicar un paradigma de programación adecuado al sistema.
- Implementar los estándares en funcionalidades reales del software.
- Garantizar buenas prácticas de desarrollo.

3. DESCRIPCIÓN DEL SOFTWARE

El sistema de parqueadero es una aplicación que permite gestionar el ingreso, permanencia y salida de vehículos dentro de un establecimiento, incluyendo:

- Registro de vehículos
- Control de tiempo de estadía
- Cálculo de tarifas
- Generación de reportes

4. DEFINICIÓN Y APLICACIÓN DEL PARADIGMA DE PROGRAMACIÓN

4.1 Definición del paradigma

Se selecciona el paradigma de Programación Orientada a Objetos (POO), el cual permite organizar el software mediante clases y objetos que representan entidades del mundo real.

4.2 Justificación

Este paradigma permite:

- Representar entidades del sistema (Vehículo, Parqueadero)
- Organizar el código de forma estructurada
- Facilitar la reutilización y mantenimiento del software

4.3 Aplicación en el sistema

Se aplican los siguientes principios:

- **Encapsulamiento:** Protección de datos dentro de las clases
- **Responsabilidad única:** Cada clase cumple una función específica
- **Separación de responsabilidades:** Organización del sistema en componentes

5. ESTÁNDARES DE CODIFICACIÓN

5.1 Convención de Nombres

Se adopta el siguiente estándar:

- Variables: camelCase
- Funciones: verbo + acción en camelCase
- Clases: PascalCase
- Constantes: MAYÚSCULAS

Ejemplo:

```
const TARIFA_POR_HORA = 2000;
```

```
let tiempoEstadia;
```

```
function calcularTarifa() {}
```

```
class Vehiculo {}
```

5.2 Reglas Obligatorias de Codificación

- Prohibido uso de nombres genéricos (ej: x, dato, var1)
- Todas las funciones deben tener una única responsabilidad
- Las variables deben ser descriptivas
- El código debe estar correctamente indentado
- Se deben validar los datos de entrada

5.3 Manejo de Errores

Se implementa validación de datos para evitar fallos en ejecución:

```
function registrarVehiculo(placa, tipo) {
  if (!placa || !tipo) {
    throw new Error("Datos incompletos del vehículo");
  }
}
```

5.4 Estructura del Proyecto

El sistema se organiza de la siguiente manera:

```
/proyecto
├── /models
├── /controllers
├── /services
├── /utils
└── app.js
```

5.5 Convenciones del Proyecto

- Archivos:
 - vehiculo.model.js
 - parqueadero.controller.js
- Sufijos:
 - .model → entidades
 - .controller → lógica

5.6 Buenas Prácticas de Desarrollo

- Código limpio y legible
- Uso de comentarios claros
- Evitar duplicidad de código
- Uso de constantes para valores fijos

6. APLICACIÓN DE ESTÁNDARES EN EL SISTEMA

6.1 Funcionalidad 1: Registro de Vehículos

```
class Vehiculo {  
    constructor(placa, tipo) {  
        this.placa = placa;  
        this.tipo = tipo;  
        this.horaIngreso = new Date();  
    }  
    obtenerInformacion() {  
        return `${this.placa} - ${this.tipo}`;  
    }  
}  
  
function registrarVehiculo(placa, tipo) {  
    if (!placa || !tipo) {  
        throw new Error("Datos inválidos");  
    }  
    const vehiculo = new Vehiculo(placa, tipo);  
    console.log("Vehículo registrado:", vehiculo.obtenerInformacion());  
    return vehiculo;  
}
```

Aplicación de estándares:

- Uso de PascalCase en la clase Vehiculo
- Uso de camelCase en variables y métodos
- Validación de datos de entrada

- Aplicación del principio de responsabilidad única

6.2 Funcionalidad 2: Cálculo de Tarifa

```
const TARIFA_POR_HORA = 2000;

function calcularTarifa(horaIngreso, horaSalida) {

  if (!horaIngreso || !horaSalida) {

    throw new Error("Fechas inválidas");

  }

  const tiempoHoras = Math.ceil((horaSalida - horaIngreso) / (1000 * 60 * 60));

  if (tiempoHoras <= 0) {

    throw new Error("Tiempo inválido");

  }

  return tiempoHoras * TARIFA_POR_HORA;

}
```

Aplicación de estándares:

- Uso de constantes en mayúsculas (TARIFA_POR_HORA)
- Validación de datos
- Función con responsabilidad específica
- Nombres descriptivos

6.3 Funcionalidad 3: Generación de Reportes

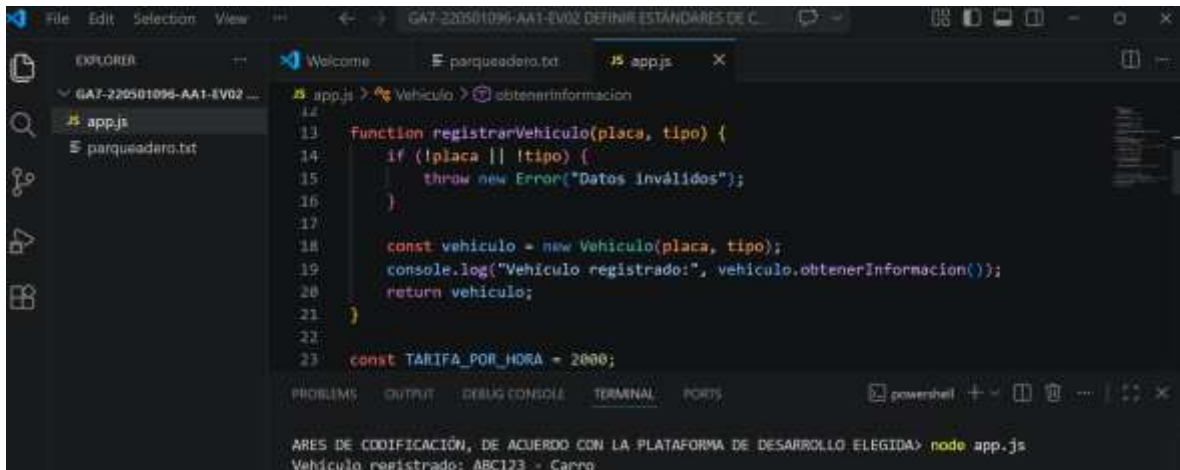
```
function generarReporte(listaVehiculos) {  
  if (!Array.isArray(listaVehiculos)) {  
    throw new Error("Dato inválido");  
  }  
  listaVehiculos.forEach(vehiculo => {  
    console.log(`Placa: ${vehiculo.placa}, Tipo: ${vehiculo.tipo}`);  
  });  
}
```

Aplicación de estándares:

- Uso de funciones con responsabilidad única
- Validación de tipo de dato
- Uso de estructuras iterativas claras
- Salida formateada

7. EVIDENCIAS (PANTALLAZOS)

Figura 1. Registro de vehículo



The screenshot shows a Visual Studio Code editor with a project named 'GA7-220501096-AA1-EV02'. The file explorer on the left shows 'app.js' and 'parqueadero.txt'. The main editor displays the 'app.js' file with the following code:

```

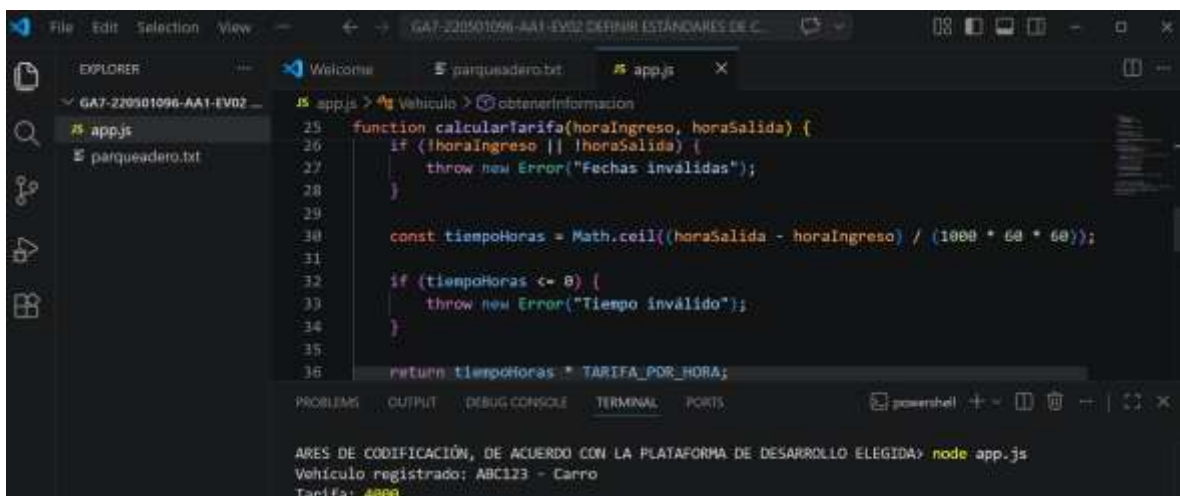
12
13 function registrarVehiculo(placa, tipo) {
14     if (!placa || !tipo) {
15         throw new Error("Datos inválidos");
16     }
17
18     const vehiculo = new Vehiculo(placa, tipo);
19     console.log("Vehículo registrado:", vehiculo.obtenerInformacion());
20     return vehiculo;
21 }
22
23 const TARIFA_POR_HORA = 2000;

```

The terminal at the bottom shows the command 'node app.js' and the output 'Vehículo registrado: ABC123 - Carro'.

Se evidencia la ejecución del sistema en la funcionalidad de registro de vehículos, donde se aplica la clase Vehiculo, la función registrarVehiculo y la validación de datos, cumpliendo con los estándares definidos.

Figura 2. Cálculo de tarifa



The screenshot shows the same Visual Studio Code editor with the 'app.js' file. The code for calculating the tariff is as follows:

```

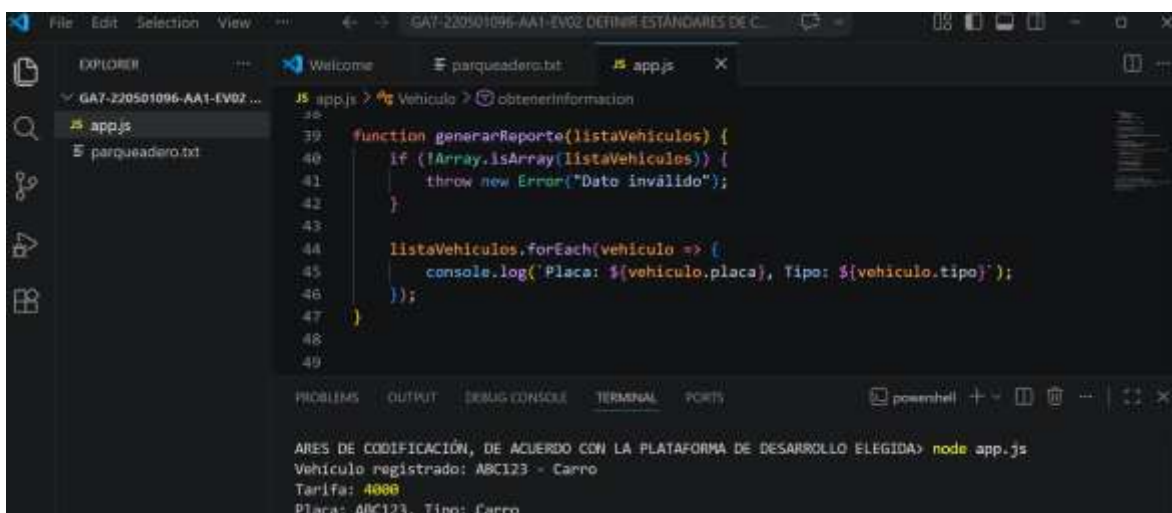
25 function calcularTarifa(horaIngreso, horaSalida) {
26     if (!horaIngreso || !horaSalida) {
27         throw new Error("Fechas inválidas");
28     }
29
30     const tiempoHoras = Math.ceil((horaSalida - horaIngreso) / (1000 * 60 * 60));
31
32     if (tiempoHoras <= 0) {
33         throw new Error("Tiempo inválido");
34     }
35
36     return tiempoHoras * TARIFA_POR_HORA;

```

The terminal at the bottom shows the command 'node app.js' and the output 'Vehículo registrado: ABC123 - Carro' and 'Tarifa: 4000'.

Se evidencia la funcionalidad de cálculo de tarifa, donde se utiliza la constante TARIFA_POR_HORA, validación de datos de entrada y lógica de negocio estructurada.

Figura 3. Generación de reporte



```
File Edit Selection View ... GA7-220501096-AA1-EV02 DEFINIR ESTANDARES DE C...
EXPLORER Welcome parqueadero.txt app.js
GA7-220501096-AA1-EV02 ...
app.js
parqueadero.txt
38
39 function generarReporte(listaVehiculos) {
40   if (!Array.isArray(listaVehiculos)) {
41     throw new Error("Data inválido");
42   }
43
44   listaVehiculos.forEach(vehiculo => {
45     console.log(`Placa: ${vehiculo.placa}, Tipo: ${vehiculo.tipo}`);
46   });
47 }
48
49
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powerhell + - |
ARES DE CODIFICACIÓN, DE ACUERDO CON LA PLATAFORMA DE DESARROLLO ELEGIDA> node app.js
Vehiculo registrado: ABC123 - Carro
Tarifa: 4000
Placa: ABC123, Tipo: Carro
```

Se evidencia la generación de reportes mediante la iteración de la lista de vehículos, aplicando buenas prácticas de codificación y salida formateada en consola.

8. CONCLUSIONES

La implementación de estándares de codificación permite desarrollar software estructurado, mantenible y escalable. La aplicación del paradigma orientado a objetos facilita la organización del sistema y la representación de entidades del mundo real.

Asimismo, la incorporación de validaciones, convenciones y buenas prácticas fortalece la calidad del código, alineándolo con entornos profesionales de desarrollo y cumpliendo de manera adecuada con los requerimientos de la evidencia.